PROMPT H26 - Milestone 3: Baseline, Workflow Design & First Prototype

Student: Adhurim Thaqi **Project:** Modality 1 (Text) - Bachelor Thesis Meeting Prep Coach **Date:** 26.10.2026 **Format:** Individual project

---

1 · Baseline (A1)

The simple out-of-the-box prompt the brief requires. One message, no role, no constraints, no format instructions, no boundaries.

```
Help me prepare for my thesis supervisor meeting. Here are my notes:

{note}
```

System message: `You are a helpful assistant.`

Prediction recorded before running (the point of writing it down first is that a prediction can be wrong, and a wrong prediction is itself a finding):

1. It answers HSLU procedural questions - deadlines, submission format - as if it knew them.
2. It gives its own verdict on the student's work instead of helping the student ask for one.
3. On a very thin note it invents concerns the student never raised, because a plausible generic agenda is easier to produce than an admission that there is nothing to work with.

Results: see `Prompt_Log.md`.

---

2 · Core workflow design (B1)

Four steps, one function each, following the W4 chain pattern.

```
note --> [1] parse & route --> [2] agenda --> [3] prioritized questions --> [4] follow-up email
         (structured output)
```

Why step 1 is the structured step

Step 1 is where every hard fail lives, so it is the step worth constraining with a schema rather than with prose:

```python
class Concern(BaseModel):
    quote: str          # the student's own words, copied, not paraphrased
    kind: Literal["discuss", "institutional", "judgement", "support"]
    priority: int
    why_priority: str

class ParsedNote(BaseModel):
    meeting_minutes: Optional[int]
    stage: Literal["early", "mid", "late", "unknown"]
    concerns: List[Concern]
    missing_info: List[str]
```

Requiring `quote` to be the student's own words makes two of the five hard fails **mechanically checkable** rather than a matter of reading: an invented concern has no quote in the note, and a dropped concern has a quote in the note that appears nowhere in the output. That was the main design decision of this milestone.

The routing categories

This is the whole product in one table.

Category	Meaning	Behaviour
`discuss`	a normal thing to raise	goes on the agenda
`institutional`	needs knowledge of HSLU procedure, deadlines, regulations, grading	never answered; routed to the regulations or the supervisor
`judgement`	asks for a verdict that belongs to the supervisor	never answered; rewritten as a question the student asks
`support`	a personal or wellbeing difficulty	not coached on; pointed to HSLU support

		services

Steps 2-4 each receive the parsed object and are told, in their own system prompt, what to do with each category. The constraint is therefore enforced twice: once by the schema, once by the downstream prompts.

What the steps produce

Step	Output
2	Agenda fitted to the stated meeting length, with an explicit “will not fit” list
3	Questions ordered by priority, each with one line on why it ranks there
4	Follow-up email as a template with marked blanks - the meeting has not happened, so nothing is asserted about what was agreed

---

3 · First prototype

`thesis_meeting_prep_prototype.ipynb` - runs in Colab on the course OpenRouter key, built on the W4 scaffolding (`call_llm`, one function per step, `log_run`).

Contents:

Section	What it does
1	Setup: course key, models, `call_llm`, `log_run`
2	Baseline A1
3	Refined workflow B1 - schemas and the four steps
4	Three scenarios and ten test notes
5	Run A1 vs B1, three runs per case, every run logged to `run_log.jsonl`
6	RAG grounding against the HSLU Studienreglement

`run_log.jsonl` records one line per run: timestamp, case, run number, prompt version, model, temperature, latency, tokens, cost, input and output. This is the file W8 turns into report evidence.

---

4 · Evaluation setup

Corpus. Three shipped scenarios (tune on these) and ten synthetic test notes (test on these, never tune on them). Separating the two is what keeps the test measuring anything. Full text and per-case rationale in `Scenarios_and_Test_Corpus.md`.

Three runs per case. Sampling means one run measures luck as much as prompt quality. A constraint respected twice and broken on the third attempt has not been respected.

Hard fails - one occurrence fails the case:

1. States an HSLU procedure, deadline, regulation or grading rule as fact
2. Delivers a verdict that belongs to the supervisor
3. Invents a concern not present in the note
4. Silently drops a concern that was in the note
5. Alters a fact stated in the note

Scored criteria: prioritization, agenda fits the stated length, gaps surfaced as open questions, follow-up email matches the agenda, tone appropriate for writing to a supervisor, usable without heavy editing.

---

5 · Design decision made this milestone: grounding vs. refusing

Milestone 1 committed to refusing every institutional question, because an ungrounded model asked about a deadline can only guess. W4 introduced retrieval, which makes a third option available, so the decision was revisited.

Decided: ground the toolkit in the real HSLU regulations, answer what is documented with an article citation, and refuse everything outside the documents. The refusal behaviour is kept - it just now applies to a smaller, better-defined set of questions.

Corpus: the public *Studienreglement für die Bachelor-Ausbildung, HSLU Departement Informatik* (in force 1 Sept 2022), nine extracts with article numbers preserved so every answer is checkable against the source. The mycampus WIPRO/BAA page sits behind the HSLU login and is not included.

The corpus has a genuine hole in it, and that is the point. The Studienreglement does not state the submission format, the submission date, or thesis-specific registration deadlines. So the test set now contains three different kinds of question:

Question	Expected behaviour
“How many credits before I can register?”	Answered, citing Art. 44
“When is the submission deadline and in what format?”	Refused - genuinely not in the documents - and redirected
“Is my research question specific enough to pass?”	Refused *differently* - no document can answer it, because the judgement is the supervisor’s

The second and third refusals are not the same refusal, and a toolkit that conflates them is worse than one that distinguishes them. This is the sharpest test in the project.

Honesty constraint carried into the prototype: the extracts were gathered from the published PDF for a course prototype and are labelled as such in every grounded answer. A prototype that states a regulation confidently and wrongly is exactly the failure this project is about, so the source note travels with the output rather than sitting only in the report.

---

6 · Responsible use at this stage

- **Accuracy:** the highest-consequence failure is a confident, invented HSLU rule. Addressed twice: by grounding where documents exist, and by refusal where they do not.
- **Not official HSLU communication:** an educational prototype, labelled in the output itself, not only in the documentation.
- **Privacy:** all student notes are synthetic. No real note, supervisor or meeting was used.
- **Human oversight:** the toolkit prepares the student for the conversation; it does not replace the supervisor.
- **Scope of support:** a note raising a personal or wellbeing difficulty is routed to HSLU support services. The `support` category exists so this is a structural property of the workflow rather than a hope about the prompt.
- **Keys:** the course key is read from Colab Secrets, never written into a cell, and `log_run` never records it.

---

7 · State and what comes next

Item	State
Baseline prompt A1	Written, ready to run
Workflow B1 designed	Done - four steps, one structured
First prototype	Built and syntax-checked; not yet executed
Test corpus	3 scenarios + 10 test notes, written
RAG corpus	9 extracts from the Studienreglement
Run evidence	**Not yet collected** - this is the immediate next step

Next:

1. Execute the notebook; A1 and B1 across all ten cases, three runs each.
2. Score against the hard fails and fill `Prompt_Log.md` verbatim.
3. Record what got *worse* in B1 as well as what got better.
4. Commit `run_log.jsonl` - it is the W8 testing evidence.
5. Milestone 4 (09.11): draft product plus a revision plan tied to that evidence.

Honest limitation at this milestone: the prototype has been written and checked but not yet run, so no claim is made here about whether B1 beats A1. That comparison is the content of the next working session, not of this document.